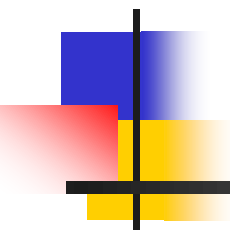


04. Java Class – a new concept



Class fundamentals

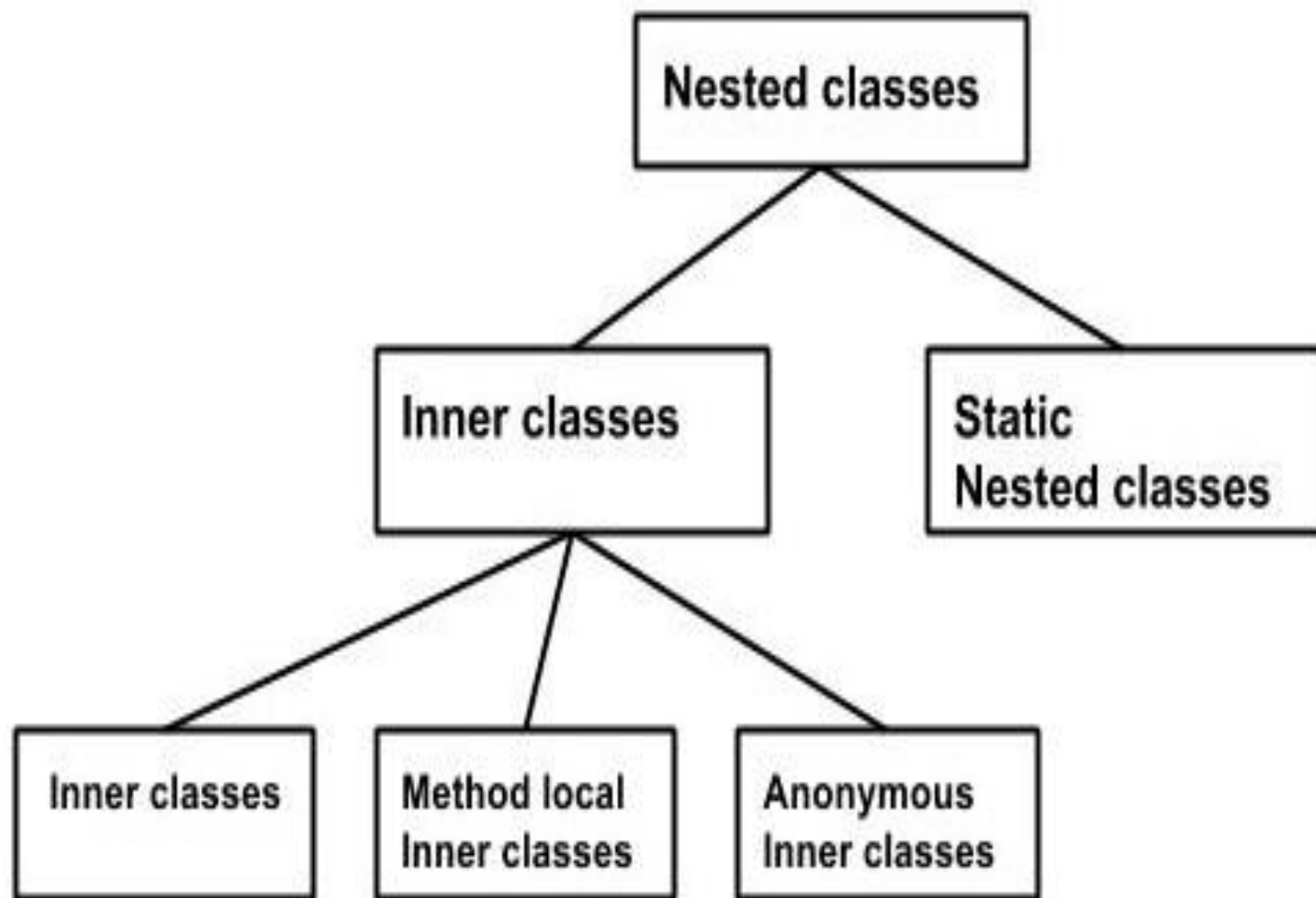
Methods

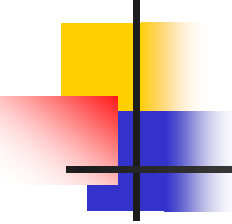
Constructors

Garbage Collection – Java Way

Access Modifiers

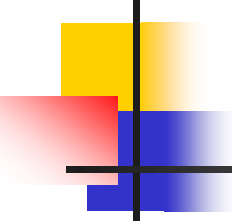
Inner Classes





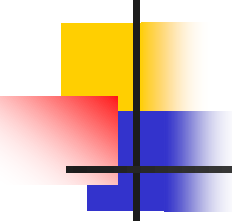
Local and Anonymous Inner Classes

- There are two additional types of inner classes. You can declare an inner class within the body of a method. Such a class is known as a *local inner class*. You can also declare an inner class within the body of a method without naming it. These classes are known as *anonymous inner classes*.



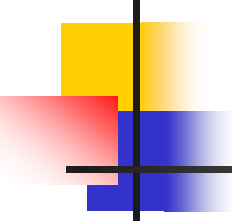
Benefits of Inner Class

- If a class is useful to only one class, it makes sense to keep it nested and together. It helps in the packaging of the classes.
- Java inner classes implements encapsulation. Note that inner classes can access outer class private members and at the same time we can hide inner class from outer world.



Benefits of Inner Class

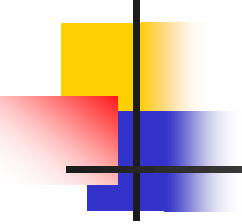
- Keeping the small class within top-level classes places the code closer to where it is used and makes the code more readable and maintainable.



Questions: Inner classes

- The program Problem.java doesn't compile. What do you need to do to make it compile? Why?

```
public class Problem {  
    String s;  
    static class Inner {  
        void testMethod() {  
            s = "Set from Inner";  
        }  
    }  
}
```

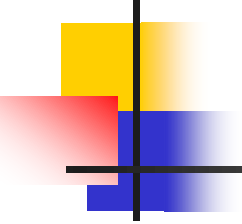


'this' keyword

Local variables

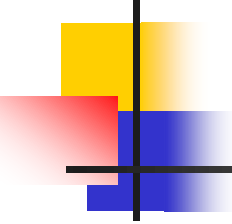
```
Box(double width, double height, double depth) {  
    this.width = width;  
    this.height = height;  
    this.depth = depth;  
}
```

Instance variables



Static Members

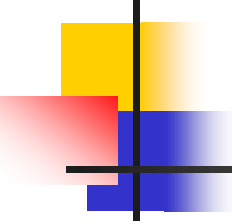
- A static class member can be accessed directly by the class name and doesn't need any object. A single copy of a static member is maintained throughout the program regardless of the number of objects created.
- Static variables are initialized only once and at the start of the execution during the lifetime of a class. These variables will be initialized first before the initialization of any instance variables.



Static Members

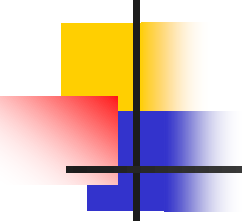
Methods declared as static (class methods) have several restrictions:

- They can only call other static methods.
- They must only access static data.
- They cannot refer to *this* or *super* in anyway.
- These methods can be accessed using the class name rather than an object reference.
- *main()* method should be always static because it must be accessible for an application to run, before any instantiation takes place.
- When *main()* begins, no objects are created, so if you have a member data, you must create an object to access it.



Static methods/Data members

```
public class Print {  
    public static String name = "default";  
  
    public static void printName()    {  
        System.out.println(name);  
    }  
  
    public static void main(String arg[]) {  
        System.out.println(Print.name);  
        Print.printName();  
    }  
}
```



```
class TrackObj
```

```
{
```

```
    //class variable
```

```
    private static int counter = 0;
```

```
    //instance variable
```

```
    private int x = 0;
```

```
    TrackObj()
```

```
    {
```

```
        counter++;
```

```
        x ++;
```

```
    }
```

```
    //member method
```

```
    public int getX()
```

```
    {
```

```
        return x;
```

```
    }
```

```
    //class method
```

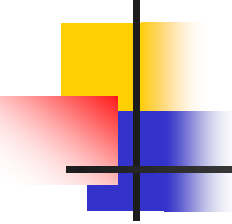
```
    public static int getCounter()
```

```
    {
```

```
        return counter;
```

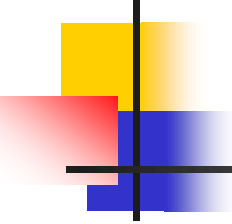
```
    }
```

```
}
```



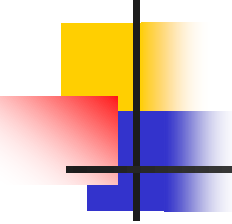
Garbage Collection

- Automatic memory deallocation is called garbage collection.
- When no references to an object exist, that object is assumed to be no longer needed, and the memory occupied by the object can be reclaimed.
- A separate low priority thread takes care of this.



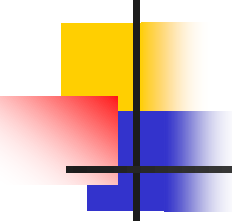
Access Modifiers

- Java provides a number of access modifiers to set the level of access for classes, fields, methods and constructors.
- A member has package or default accessibility when no accessibility modifier is specified.
- **Access Modifiers:**
 1. private
 2. protected
 3. default
 4. public



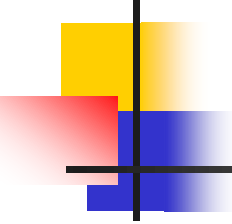
private access modifier

- The *private* (most restrictive) access modifier is used for fields or methods and cannot be used for classes and Interfaces.
- It also cannot be used for fields and methods within an interface.
- Field, method declared private are strictly controlled, and that member can be accessed only by other members of that class.
- A standard design strategy is to make all fields private and provide public getter methods for them.



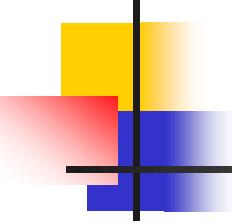
protected access modifier

- Discussed later under the topic - Inheritance



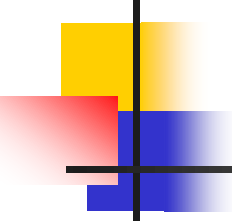
default access modifier

- Java provides a default access modifier which is used when no access modifier is specified.
- Any class, field, method or constructor that has no declared access modifier is accessible only by classes in the same package.
- The default modifier is not used for fields and methods within an interface.



public access modifier

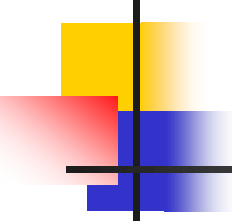
- Fields, methods and constructors declared public (least restrictive) within a public class are visible to any class in the Java program, whether these classes are in the same package or in another package.



Access Levels

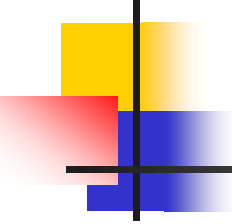
The following table shows the access to members permitted by each modifier.

Modifier	Class	Package	Subclass	World
<i>public</i>	Y	Y	Y	Y
<i>protected</i>	Y	Y	Y	N
<i>default</i>	Y	Y	N	N
<i>private</i>	Y	N	N	N



Enum Types

- An enum type is a special data type that enables for a variable to be a set of predefined constants. The variable must be equal to one of the values that have been predefined for it. Common examples include compass directions (values of NORTH, SOUTH, EAST, and WEST) and the days of the week.



Enum Types

- In the Java programming language, you define an enum type by using the enum keyword. For example, you would specify a days-of-the-week enum type as:

```
public enum Day {  
    SUNDAY, MONDAY, TUESDAY, WEDNESDAY,  
    THURSDAY, FRIDAY, SATURDAY  
}
```